

第3章 流程控制

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 if/else 的 boolean 强约束,理解"悬挂 else"规则与 C 的差异
- 掌握 switch 的两代语法:传统 fallthrough 版与箭头版,以及 C 的 switch 三个缺陷如何被修复
- 掌握 switch 表达式与 yield,学会用表达式替换 if-else 链
- 掌握 while/do-while/for 三种循环与增强 for,能与 C 的三种循环逐条对照
- 掌握 break/continue 与带标签跳转,知道 Java 的 goto 是保留字
- 能写出菜单驱动的控制台程序,正确组合循环与跳转
- 理解循环变量捕获(effectively final)约束,避免匿名类与 lambda 的编译错误

💡 从 C 到 Java:本章主题如何迁移

流程控制在 Java 里"形似而神不似":语句长相几乎相同,但约束与陷阱完全不同。

第一,条件必须是 boolean。C 的 `if (x)` 接受任意表达式,0 为假、非 0 为真;Java 只接受 boolean。`if (x = 5)` 在 C 里是经典 bug,在 Java 里是编译错误。代价:写比较时多敲两下;收益:"赋值当比较"这类事故从语言层面灭绝。

第二,switch 被彻底重做。C 的 switch 有三大缺陷:只能整型、默认 fallthrough、无作用域隔离(变量声明容易串)。Java 首先放开类型(字符串、枚举),然后用箭头语法 `case x ->` 消灭 fallthrough,最后把 switch 升级为"表达式"(可以给变量赋值,还能 yield 返回值)——现代 Java 的 switch 已经是一种**模式匹配**工具(第14章),与 C 的"跳转表"完全是两个物种。

第三,goto 的态度。C 的 goto 用得好可以简化错误处理,用不好就是意大利面;Java 干脆把 `goto` 定为保留字禁止使用,用**带标签的 break/continue**覆盖"跳出多层循环"这一 goto 的正当用途。想做的事一样,工具换成了安全的。

本章策略:C 有的先对照(if/while/for 语法几乎零成本迁移),然后重点掌握 switch 的新语法——这是本章最值钱的部分。

📦 前置知识自查

- C 的 if/else、while、do-while、for 语法与 break/continue
- C 的 switch-case 与 fallthrough 行为(忘记 break 的后果)
- C 的三目表达式与常见循环模式(计数器、哨兵值、遍历数组)
- 第1章知识:boolean 类型、枚举 enum、数组与 length 属性

🚀 本章学习路线

1. **第一阶段(约 1 小时,快扫):**§3.1 与 §3.3 与 C 几乎相同,对照表格过一遍即可,重点看"差异"列
2. **第二阶段(约 1.5 小时,攻坚):**§3.2 的 switch 三讲(传统版、箭头版、表达式版)逐段运行,对比 C 的写法;§3.4 的标签跳转做一遍二维查找
3. **第三阶段(实验,约 1 小时):**独立完成章末实验"菜单驱动的数据统计程序",用自测题验收
4. **验收标准:**能不看资料写出:箭头版 switch、switch 表达式(含 yield)、带标签 break 的二维查找

本章知识导图

- §3.1 if/else:boolean 强约束 / 悬挂 else / if-else 链
- §3.2 switch:从 C 的缺陷到 Java 的新语法(传统版 / 箭头版 / switch 表达式 / 字符串与枚举)
- §3.3 三种循环:while / do-while / for / 增强 for / 循环变量捕获
- §3.4 break、continue 与带标签跳转(goto 保留字 / 跳出多层循环)
- §3.5 无限循环写法与生产实践(菜单驱动 / 分支策略选型)

§3.1 if/else:条件必须是 boolean

3.1.1 与 C 的对照

语法与 C 完全相同: `if (条件) 语句; else 语句;`。唯一差异:条件必须是 `boolean` 表达式。`if (1)`、`if (ptr)`、`if (n)` 在 Java 里全部编译失败,必须显式写成 `if (n != 0)`、`if (ptr != null)`。这带来一个连锁好处:`if (n = 5)` 不可能通过编译——因为 `n = 5` 的类型是 `int` 不是 `boolean`,编译器直接拦下。

示例 3-1 if/else 与 boolean 条件

```
public class IfDemo {
    public static void main(String[] args) {
        int score = 75;
        // C: if (score) 合法;Java:必须显式比较
        if (score >= 60) {
            System.out.println("及格");
        } else {
            System.out.println("不及格");
        }
        // if (score) {}           // 编译错误:不是 boolean
        // if (score = 60) {}      // 编译错误:赋值表达式是 int

        // 三目与 if 等价(C 程序员熟知的写法)
        String level = score >= 60 ? "及格" : "不及格";
        System.out.println(level);
    }
}
```

3.1.2 悬挂 else 与 if-else 链

悬挂 `else`(dangling else)规则与 C 相同:`else` 与最近的未配对 `if` 结合。Java 的缩进规范(官方风格 4 空格)要求 `if/else` 必须带花括号,从格式上消灭了悬挂 `else` 的阅读歧义。多分支时用 `if-else` 链;分支多(≥ 4)且是"单值分派"时,应该改用 §3.2 的 `switch`——这是本章最重要的工程判断。

主题	C 的写法	Java 的写法
if 条件	任意数值,0 为假非 0 为真	必须 <code>boolean</code> ; <code>if (x)</code> 报编译错误
赋值当比较	<code>if (n = 5)</code> 合法,经典 bug	编译错误,无法写出
悬挂 else	与最近 <code>if</code> 结合(需小心)	规则相同,但规范强制花括号
多分支	<code>if-else</code> 链或 <code>switch</code>	<code>if-else</code> 链、 <code>switch</code> 或 <code>switch</code> 表达式

⚠ 易错点 1:else 就近配对

`if (a) if (b) x(); else y();` 中 `else` 属于内层 `if (b)` 还是外层? 属于内层 `if (b)` (就近配对), 与 C 相同。想要 `else` 属于外层, 必须写 `if (a) { if (b) x(); } else y();`。Java 官方风格强制花括号, 这类歧义在规范代码中不存在, 但阅读别人的“紧凑代码”时仍要警惕。

§3.2 switch:从 C 的缺陷到 Java 的新语法

3.2.1 回顾 C 的 switch 三大缺陷

C 的 `switch` 是跳转表:① 表达式只能是整型(不能是字符串);② **fallthrough**——漏写 `break` 会一路执行下去;③ `case` 里的变量声明作用域混乱。Java 的演进分三步:先放开类型(Java 5 枚举、Java 7 字符串),再引入箭头语法消灭 `fallthrough`(Java 14),最后升级为表达式(Java 14,支持 `yield` 与模式匹配)。

3.2.2 传统写法(与 C 对照)

示例 3-2 传统 `switch`:必须 `break`(与 C 相同)

```
public class SwitchOldDemo {
    public static void main(String[] args) {
        int day = 3;
        String name;
        switch (day) {
            case 1:
                name = "周一";
                break;           // 漏掉 break 就 fallthrough(与 C 相同)
            case 2:
                name = "周二";
                break;
            case 3:
                name = "周三";
                break;
            default:
                name = "未知";
        }
        System.out.println(name);    // 周三

        // 字符串与枚举也可以 switch(Java 7/5 起)
        String fruit = "apple";
        switch (fruit) {             // 内部用 equals 比较
            case "apple":
                System.out.println("苹果");
                break;
            case "banana":
                System.out.println("香蕉");
                break;
            default:
                System.out.println("其他水果");
        }
        // 枚举 switch:case 不用写枚举名
        // switch (season) { case SPRING: ... }
    }
}
```

⚠ 易错点 2:忘记 break 的 fallthrough

C 的经典事故在 Java 里同样存在(传统写法下): `case` 执行完没有 `break`, 会继续执行下一个 `case`。生产代码建议:新代码一律用下面的箭头语法,编译器会强制你为每个 `case` 写语句, `fallthrough` 从机制上消失。

3.2.3 箭头语法(Java 14+):推荐写法

示例 3-3 箭头版 switch:无 fallthrough

```
public class SwitchNewDemo {
    public static void main(String[] args) {
        int day = 3;
        String name;
        switch (day) {
            case 1 -> name = "周一";           // 箭头后是单条语句,无需 break
            case 2 -> name = "周二";
            case 3 -> name = "周三";
            default -> name = "未知";
        }
        System.out.println(name);

        // 多值标签:一个 case 匹配多个值(逗号分隔)
        String season = "夏";
        switch (season) {
            case "春", "夏" -> System.out.println("温暖");
            case "秋", "冬" -> System.out.println("凉爽");
            default -> System.out.println("未知季节");
        }
        // 箭头后可以是代码块(需要多条语句时)
        switch (day) {
            case 1 -> {
                System.out.println("周一");
                System.out.println("新的一周");
            }
            default -> System.out.println("其他");
        }
    }
}
```

3.2.4 switch 表达式:switch 可以有返回值

示例 3-4 switch 表达式与 yield

```

public class SwitchExprDemo {
    public static void main(String[] args) {
        int day = 3;
        // switch 表达式:整体是一个值,可以直接赋值
        String name = switch (day) {
            case 1 -> "周一";
            case 2 -> "周二";
            case 3 -> "周三";
            default -> "未知";
        };
        System.out.println(name);    // 周三

        // 需要多条语句的分支:用 yield 返回值
        int month = 2;
        int days = switch (month) {
            case 1, 3, 5, 7, 8, 10, 12 -> 31;
            case 4, 6, 9, 11 -> 30;
            case 2 -> {
                int leap = (month % 4 == 0) ? 29 : 28;    // 简化,勿用于真实历法
                yield leap;                            // yield 返回值
            }
            default -> -1;
        };
        System.out.println(days);    // 28

        // 枚举 switch 表达式:穷尽所有枚举值时可以省略 default
        // Season s = Season.SPRING;
        // String desc = switch (s) {
        //     case SPRING -> "春";
        //     case SUMMER -> "夏";
        //     case AUTUMN -> "秋";
        //     case WINTER -> "冬";
        // };    // 枚举分支齐全,无需 default,编译器能验证穷尽性
    }
}

```

★ 重点:switch 表达式的三条铁律

- 表达式版本 必须穷尽 :所有可能值都有分支,否则加 default(枚举全列完可省略,编译器可验证)
- 每个分支必须"产生一个值":箭头后是表达式,或代码块以 yield 结尾
- 箭头版不会 fallthrough: case 之间不需要 break,多标签用逗号 ——与 C 的心智模型完全不同

🚀 工程建议:分支选型三问

写分支前问三个问题: ① 是"单值分派"吗?(按一个变量的取值走不同逻辑)→ 用 switch; ② 是"范围/复合条件"吗?(score >= 60 这种)→ 用 if-else 链; ③ 是"类型分派"吗?(按对象类型走不同逻辑)→ 用 instanceof 模式匹配(第2章)或第14章的模式匹配 switch。选对工具比写对语法更重要。

§3.3 三种循环:while、do-while 与 for

3.3.1 与 C 逐条对照

Java 的三种循环语法与 C 几乎逐字相同: while (条件)、do 语句 while (条件);、for (初始化; 条件; 更新)。差异有三: ① 条件必须是 boolean; ② for 的循环变量可以声明为 var; ③ 新增增强 for(遍历数组与集合)。do-while 在 Java 里仍然存在(区别于 Go/Rust 等去掉了它的语言)。

场景	C 的写法	Java 的写法
计数循环	for (i = 0; i < n; i++)	for (int i = 0; i < n; i++)(条件须 boolean)
条件循环	while (n > 0)	while (n > 0)(相同)
至少执行一次	do { } while (...)	do { } while (...)(相同)
遍历数组	for (i = 0; i < N; i++) 用 a[i]	for (int x : a)(增强 for,无需下标)
遍历需要下标	同上	用传统 for 或 IntStream.range(第4章)

示例 3-5 三种循环与增强 for

```
public class LoopDemo {
    public static void main(String[] args) {
        // while:经典"哨兵值"模式(C 相同)
        int sum = 0, n = 1;
        while (n <= 100) {
            sum += n;
            n++;
        }
        System.out.println("1..100 之和 = " + sum);    // 5050

        // do-while:至少执行一次(用户输入校验)
        int input;
        do {
            System.out.print("请输入 1~7: ");
            input = 7;           // 真实代码里从控制台读
        } while (input < 1 || input > 7);
        System.out.println("OK: " + input);

        // for:与 C 完全一致;变量可声明为 var
        for (int i = 0; i < 5; i++) {
            System.out.print(i + " ");
        }
        System.out.println();

        // 增强 for:遍历数组(不需要下标时首选)
        int[] scores = {85, 92, 78, 66, 99};
        int total = 0;
        for (int s : scores) {
            total += s;
        }
        System.out.println("平均分 = " + (double) total / scores.length);
    }
}
```

3.3.2 循环变量捕获:effectively final

在 C 里,循环变量可以随便被内层函数取地址;Java 的 lambda 与匿名类(第4、6章)捕获局部变量时有一个约束: 被捕获的变量必须是 final 或"实际上的 final"(赋值后不再改变)。经典写法是把循环变量复制一份:

示例 3-6 循环变量捕获

```
import java.util.ArrayList;
import java.util.List;

public class CaptureDemo {
    public static void main(String[] args) {
        List<Runnable> tasks = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            int copy = i;                // effectively final 的副本
            tasks.add(() -> System.out.print(copy + " "));
        }
        for (Runnable t : tasks) {
            t.run();                    // 0 1 2
        }
        System.out.println();
        // 若直接捕获 i:编译错误 "local variables referenced from
        // a lambda expression must be final or effectively final"
    }
}
```

⚠ 易错点 3:lambda 捕获循环变量

很多语言(如 C 的闭包扩展、JS 的 var)在循环里捕获循环变量会得到"全部是最后一个值";Java 的解法更彻底:捕获必须 **effectively final**,非法的直接编译失败。惯例:循环内定义 `int copy = i;` 再捕获。这个约束也顺带防止了"共享可变量"这一并发 bug 温床。

§3.4 break、continue 与带标签跳转

3.4.1 单层跳转:与 C 相同

`break` 跳出当前循环或 `switch`; `continue` 跳过本轮进入下一轮——语义与 C 完全一致。Java 的 `continue` 不能用于 `switch`(会报错,因为 `switch` 不是循环)。

3.4.2 带标签的 break/continue:goto 的替代品

`goto` 是 Java 的保留字,不可使用。跳出多层嵌套循环的正当需求由带标签的 **break/continue** 承担:给外层循环起名 `outer:`,内层写 `break outer;` 直接跳出整个外层循环;`continue` 带标签则跳到外层循环的下一轮。经典场景:二维数组查找。

示例 3-7 二维查找与带标签 break

```

public class LabelDemo {
    public static void main(String[] args) {
        int[][] grid = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        int target = 5;
        boolean found = false;

        outer: // 标签:给外层循环命名
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[i].length; j++) {
                if (grid[i][j] == target) {
                    System.out.println("找到 " + target + " 在 (" + i + ", " + j + ")");
                    found = true;
                    break outer; // 跳出两层循环
                }
            }
        }
        if (!found) {
            System.out.println("未找到");
        }
        // 普通 break 只能跳出一层,内层循环结束后外层还会继续
        // 需要"跳过外层本轮剩余部分"时用 continue outer;
    }
}

```

★ 重点:标签跳转的使用纪律

带标签跳转只用于"跳出多层嵌套"这一种场景,且标签名建议用英文小写(如 `outer:`、`search:`)。不要用它实现"任意跳转"——那等于复活 `goto`,可读性灾难。多层嵌套本身也说明函数太长,重构(提取方法 + 提前 `return`)往往是更优解。

§3.5 无限循环与生产实践

3.5.1 无限循环写法对照

风格	C 的写法	Java 的写法
经典无限循环	<code>while (1) { }</code>	<code>while (true) { }</code> (或 <code>for(;;)</code>)(1 不是 boolean!)
服务主循环	<code>for (;;) { ... }</code>	<code>while (true) { ... }</code> (等价)
退出方式	<code>break / goto / return</code>	<code>break / return / 抛异常</code> (第9章)

3.5.2 菜单驱动的控制台程序:流程控制的集大成

生产中最常见的"流程控制全家桶"就是菜单程序:`while` 包住整个交互循环,`switch` 分派菜单项,`do-while` 校验输入,`for`/增强 `for` 处理数据,`break`/`return` 退出。第1章的 `Scanner` 用法在这里继续使用。

示例 3-8 菜单程序骨架(实验题的最小版本)


```
import java.util.Scanner;

public class MenuSkeleton {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        while (true) { // 主循环:菜单反复出现
            System.out.println("1. 录入 2. 统计 3. 退出");
            System.out.print("请选择: ");
            int choice = Integer.parseInt(in.nextLine());
            switch (choice) {
                case 1 -> System.out.println("录入功能");
                case 2 -> System.out.println("统计功能");
                case 3 -> {
                    System.out.println("再见");
                    return; // 结束整个程序
                }
                default -> System.out.println("无效选项");
            }
        }
    }
}
```

工程建议:交互输入与健壮性

真实生产里交互程序注意三点: ① 解析输入必须处理异常 (Integer.parseInt 遇到非数字会抛异常,第9章学习完整处理); ② 不要用 Scanner 混用 nextLine 与 nextInt (换行符残留问题),统一用 nextLine 再解析; ③ 循环必须有明确的退出路径,菜单程序用 return/break,服务程序用关闭标志位——无限循环本身不是问题,没有出口才是。

本章速查表

主题	要点 / 写法
if 条件	必须 boolean;if (x) 编译错误,写 if (x != 0)
悬挂 else	else 就近配对;官方规范强制花括号
传统 switch	case 后必须 break,否则 fallthrough(与 C 相同);可 switch 字符串/枚举
箭头 switch	case x -> 语句;无 fallthrough;多标签 case 1, 2 -> ...(Java 14+)
switch 表达式	switch (x) { ... } 整体是值;多语句分支用 yield;必须穷尽
枚举 switch	case 不写枚举名;分支全列完可省略 default(编译器验证穷尽)
while	与 C 相同;条件必须 boolean
do-while	至少执行一次,与 C 相同(区别于 Go/Rust/Python)
for	for (int i = 0; i < n; i ++);循环变量可 var
增强 for	for (int x : arr) 遍历数组/集合;需要下标时用传统 for
break/continue	单层语义与 C 相同;continue 不能用在 switch
标签跳转	outer: 标签 + break outer;;continue outer;;goto 是保留字
循环捕获	lambda 捕获的变量必须 effectively final;用副本 int copy = i
无限循环	while (true) 或 for(;;);必须有退出路径
分支选型	单值分派用 switch;范围条件用 if-else;类型分派用 instanceof/模式匹配

最小必会清单

- 能解释 if 的 boolean 强约束如何消灭"赋值当比较"bug,并写出等价写法

- 能默写箭头版 switch 与 switch 表达式(含 yield 与多标签)
- 能说出 C 的 switch 三大缺陷及 Java 的修复方式
- 能写出三种循环与增强 for,并说明 do-while 的存在价值(至少执行一次)
- 能写出带标签 break 的二维查找;能说出 goto 为什么不能用
- 能解释 lambda 捕获循环变量的 effectively final 约束与副本技巧
- 能写出 while(true) + switch 的菜单骨架,并设计明确的退出路径
- 能根据"单值/范围/类型"三种场景选择 if、switch 或 instanceof

自测题

Q 1. 下列哪些行能通过编译? if (1) { }, if (flag) { }, if (s.length() > 0) { }, while (n) { }

✅ 参考答案

第二、三行通过; if (1) 与 while (n) 编译失败(1 和 n 不是 boolean)。正确写法 if (n != 0)。这正是"把 C 的真值规则删掉"的结果。

Q 2. 下面的输出是什么? int day = 2; switch (day) { case 1 -> System.out.print("A"); case 2 -> System.out.print("B"); default -> System.out.print("C"); }

✅ 参考答案

只输出 "B"。箭头语法没有 fallthrough,每个分支执行完自动结束,无需 break。若换成传统写法且 case 2 后漏写 break,会输出 "BC"。

Q 3. 用 switch 表达式写出:输入数字 1~3,返回 "低/中/高",其他返回 "未知"。

✅ 参考答案

```
String level = switch (n) {
    case 1 -> "低";
    case 2 -> "中";
    case 3 -> "高";
    default -> "未知";
};
```

Q 4. 这段代码能编译吗?为什么? for (int i = 0; i < 3; i++) { tasks.add(() -> System.out.println(i)); }

✅ 参考答案

不能编译: i 在 lambda 中被引用但会变化,不是 effectively final。修正: int copy = i; 后捕获 copy。这是 Java 与 C(闭包扩展)最大的不同:编译期就禁止捕获可变循环变量。

Q 5. 下面的代码找得到 8 吗? int[][] g = {{1,2},{3,4}}; outer: for (int i = 0; i < 2; i++) { for (int j = 0; j < 2; j++) { if (g[i][j] == 8) break outer; } } 如果找不到,输出什么?

✅ 参考答案

找不到(数组中根本没有 8),但代码不会报错:两层循环都正常结束,break outer 从未执行。若找到则立即跳出两层——带标签 break 只做"跳转",不做"是否存在"的判断,所以需要 found 标志配合,如示例 3-7。

Q 6. 如何判断一个循环是不是"死循环"?给出两个能保证退出的循环写法。

✅ 参考答案

循环体内没有任何 `break/return/异常出口`,且条件恒真,就是死循环。保证退出的写法:`while (true) { if (exit) break; ... }` (标志位 + `break`);或 `for (;;) { if (cond) return; }` (直接返回)。菜单程序用 `return` 退出主循环是最清晰的。

章末实验题:菜单驱动的数据统计程序

📁 实验 3:菜单驱动的数据统计程序

实验目标:编写一个菜单驱动的控制台程序:录入整数、素数筛选、统计汇总,覆盖本章全部流程控制知识点。

实验要求:

- 任务 1:主菜单用 `while (true)` 无限循环 + 箭头版 `switch` 分派,含 `default` 与退出分支(覆盖 §3.5 无限循环、§3.2 箭头 `switch`)
- 任务 2:录入功能用 `do-while` 至少读一次,遇到哨兵值 `-1` 用 `break` 结束(覆盖 §3.3 `do-while`、§3.4 `break`)
- 任务 3:素数筛选用双层 `for` (试除法),内层用 `continue` 跳过非候选、`break` 提前结束(覆盖 §3.3 `for`、§3.4 `continue/break`)
- 任务 4:统计功能用增强 `for` 遍历,遇到负数用带标签 `break outer` 中止并提示(覆盖 §3.3 增强 `for`、§3.4 标签跳转)
- 任务 5:`if-else` 做输入校验与边界判断(空数据、无效选项)(覆盖 §3.1 `if/else`)
- 任务 6:退出菜单用 `return`,录入与统计共用同一个数组(覆盖 §3.4 `return`、数组与枚举基础)

实验环境:JDK 17+; `javac MenuApp.java` 编译, `java MenuApp` 运行。

第一步 搭骨架:main 里的 `while(true) + switch`,四个菜单项各一个静态方法,编译通过再填空

第二步 现录入:`do-while + 哨兵`;注意数组容量上限(`count < data.length` 也要检查)

第三步 现素数筛选与统计:先写普通版本跑通,再为统计加上带标签 `break` 的"遇到负数中止"逻辑,验证标签跳转的效果

✅ 参考答案要点

```

import java.util.Scanner;

/**
 * 菜单驱动的数据统计程序 — 第3章综合实验参考答案
 * 覆盖:while(true)、箭头 switch、do-while、for、增强 for、
 * break、continue、带标签 break、if-else、return
 */
public class MenuApp {
    private static final Scanner in = new Scanner(System.in);
    private static final int MAX = 100;

    public static void main(String[] args) {
        int[] data = new int[MAX];
        int count = 0;

        while (true) {
            // §3.5:主菜单无限循环
            System.out.println("==== 数据统计菜单 =====");
            System.out.println("1. 录入数据(以 -1 结束)");
            System.out.println("2. 素数筛选");
            System.out.println("3. 统计汇总");
            System.out.println("4. 退出");
            System.out.print("请选择(1-4): ");
            int choice = readInt(); // 无效输入返回 -1

            if (choice == -1) {
                // §3.1:输入非法,重新显示菜单
                System.out.println("输入无效,请输入数字 1-4");
                continue; // §3.4:continue 回到菜单
            }
            switch (choice) {
                // §3.2:箭头 switch,无 fallthrough
                case 1 -> count = inputData(data);
                case 2 -> printPrimes(data, count);
                case 3 -> printStats(data, count);
                case 4 -> {
                    System.out.println("再见!");
                    return; // §3.4:return 结束整个程序
                }
                default -> System.out.println("选项无效(1-4)");
            }
        }
    }

    // 读取一个整数;非法输入返回 -1(简化:异常先不处理,第9章详讲)
    static int readInt() {
        try {
            return Integer.parseInt(in.nextLine().trim());
        } catch (NumberFormatException e) {
            return -1;
        }
    }

    // §3.3 do-while:至少执行一次;哨兵 -1 用 break 结束
    static int inputData(int[] data) {
        int count = 0;
        int n;
        do {
            System.out.print("整数(输入 -1 结束): ");
            n = readInt();
            if (n == -1) {
                break; // §3.4:break 跳出 do-while
            }
            data[count++] = n; // 容量检查简化,MAX=100
        } while (count < MAX);
        System.out.println("已录入 " + count + " 个数");
        return count;
    }

    // §3.3 for + §3.4 continue/break:试除法筛素数
    static void printPrimes(int[] data, int count) {

```

```

    if (count == 0) { // §3.1 边界判断
        System.out.println("没有数据");
        return;
    }
    boolean found = false;
    for (int i = 0; i < count; i++) {
        int v = data[i];
        if (v < 2) {
            continue; // 跳过非候选:0、1、负数
        }
        boolean prime = true;
        for (int d = 2; d * d <= v; d++) { // 试除到 sqrt(v) 即可
            if (v % d == 0) {
                prime = false;
                break; // 已确认合数,提前退出内层
            }
        }
        if (prime) {
            System.out.print(v + " ");
            found = true;
        }
    }
    System.out.println(found ? "(以上为素数)" : "(没有素数)");
}

// §3.3 增强 for + §3.4 带标签 break:统计遇到负数立即中止
static void printStats(int[] data, int count) {
    if (count == 0) {
        System.out.println("没有数据");
        return;
    }
    int sum = 0, max = Integer.MIN_VALUE, min = Integer.MAX_VALUE;
    int skipped = 0;
    outer: // §3.4: 标签跳转(goto 的替代)
    for (int v : data) { // 增强 for:遍历整个数组
        if (v < 0) {
            skipped++;
            continue; // 负数跳过统计,继续下一轮
        }
        sum += v;
        max = Math.max(max, v);
        min = Math.min(min, v);
        if (skipped > 3) {
            System.out.println("负数过多,统计中止");
            break outer; // 跳出整个循环
        }
    }
    if (skipped > 3) {
        System.out.println("本次统计无效,请检查数据");
        return;
    }
    int valid = count - skipped;
    System.out.printf("有效数据 %d 个:和=%d 平均=%.1f 最大=%d 最小=%d\n",
        valid, sum, (double) sum / valid, max, min);
}
}

```

设计说明:每个菜单项对应一个方法,main 只做"循环 + 分派",这是菜单程序的标准结构。switch (choice) 用箭头语法,四个分支互不干扰;输入非法时 continue 直接回到菜单顶部,不进入 switch。素数筛选体现"continue 跳过 + break 提前终止"的组合;统计功能用带标签 break 演示"数据异常时整体中止"——这正是 C 里 goto 的经典用途,Java 用标签实现了它。扩展方向:数据存文件(第7章)、多线程统计(第9章)。

下一章预告:第4章 函数与方法。流程控制让程序"走起来",方法让程序"组织起来"——Java 的方法与 C 函数有何异同、参数到底怎么传、lambda 与 Stream 如何改写你的编程习惯。